

TP: Breaking the Classics – Answers

Imane Fouad, UM6P – Intro to Cryptography

Warm-Up

1. Encrypt (K=4)

Ciphertext: GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD

```
def CaesarCipher(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            shift_base = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
        else:
            result += char
    return result

string="CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ"
print("Ciphertext:", CaesarCipher(string, 4))
```

2. Decrypt (K=8)

Plaintext: YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS

```
def CaesarDecipher(text, shift):
    return CaesarCipher(text, -shift)

cipher="GWCL JMBBMZ VWB JM CAQVO BWTA BW AWTDM BPQA"
print("Decrypted:", CaesarDecipher(cipher, 8))
```

3. Brute Force

26 possible keys, so all shifts are tried. The plaintext is found at shift 15:

THE KEY IS UNKNOWN TRY BRUTE FORCE

```
def CaesarbruteForce(text):
    for shift in range(26):
        print(f"Shift {shift}: {CaesarCipher(text, shift)}")

cipher2="ESP VPJ TD FYVZYHY ECJ MCFEP QZCNP"
CaesarbruteForce(cipher2)
```

Room 1: The Letter Wall

1. Encrypt

Using the given substitution table:

KEEP THE SECRET SAFE → LTTN EKT YTJRTE YQGT

2. Decrypt

ERVYE DXVR SXFPJ → TRUST YOUR LOGIC

3. Frequency analysis

An attacker compares the ciphertext letter frequencies to the known English frequencies (E, T, A, O, I, N...) and matches the most frequent ciphertext letters to the most frequent plaintext letters to guess part of the key.

```
def count_occurrences(text):
    occurrences = {}
    for char in text:
        if char.isalpha():
            char = char.lower()
            occurrences[char] = occurrences.get(char, 0) + 1
    return occurrences

print("Letter frequencies in ciphertext:")
print(sorted(count_occurrences(text).items(), key=lambda x: x[1], reverse=True))
```

4. Break the intercepted message

Using the frequency mapping (E→t, T→k, R→t... built from the hint order) the message is recovered:

"the secret message is hidden in this paragraph. letter frequency does not reveal the whole key, but it gives a good start. look for common words, repeated patterns, and short words. when you recover the message, the vigenere key has four letters."

```
def recover_text(text):
    text = text.lower()
    recovered_text = ""
    for char in text:
        if char=='e':
            recovered_text += 't'
        elif char=='k':
            recovered_text += 'h'
        elif char=='t':
            recovered_text += 'e'
        elif char=='p':
            recovered_text += 'i'
        elif char=='y':
            recovered_text += 's'
        elif char=='x':
            recovered_text += 'o'
        elif char=='j':
            recovered_text += 'c'
        elif char=='q':
            recovered_text += 'a'
        elif char=='o':
            recovered_text += 'n'
        elif char=='z':
            recovered_text += 'd'
        elif char=='n':
            recovered_text += 'p'
        elif char=='f':
            recovered_text += 'g'
        elif char=='b':
            recovered_text += 'm'
        elif char=="i":
            recovered_text += 'w'
        elif char=="h":
            recovered_text += 'v'
        elif char=="s":
            recovered_text += 'l'
        elif char=="m":
            recovered_text += 'r'
```

```

        recovered_text += 'b'
    elif char=="v":
        recovered_text += 'u'
    elif char=="d":
        recovered_text += 'y'
    elif char=="l":
        recovered_text += 'k'
    elif char=="g":
        recovered_text += 'f'
    else:
        recovered_text += char
    return recovered_text

print("Recovered text:", recover_text(text))

```

The clue tells us the Vigenere key has 4 letters.

Room 2: Break the Vigenere Cipher

1. Recover the key

The ciphertext is split into 4 groups (one per key letter). For each group, the most frequent ciphertext letter is assumed to map to plaintext E, which gives the Caesar shift and therefore the key letter.

Recovered key: CODE

```

def vigenere_find_key(ciphertext, key_length):
    key = ""
    for i in range(key_length):
        nth_chars = ciphertext[i::key_length]
        frequencies = count_occurrences(nth_chars)
        most_common_letter = sorted(frequencies.items(), key=lambda x: x[1], reverse=True)[0][0]
        shift = (ord(most_common_letter) - ord('e')) % 26
        key += chr(shift + ord('a'))
    return key

print("Vigenere Key:", vigenere_find_key(Ciphertext, 4))

```

2. Decrypt the message

```

def vigenere_decrypt(ciphertext, key):
    decrypted_text = ""
    key_length = len(key)
    count=0
    for char in ciphertext:
        if char.isalpha():
            shift = ord(key[count % key_length])-ord('a')
            decrypted_text += CaesarDecipher(char, shift)
            count += 1
        else:
            decrypted_text += char
    return decrypted_text

print("Decrypted Text:", vigenere_decrypt(Ciphertext, "code"))

```

THE SECRET KEEPER LEFT THESE NOTES BECAUSE THE LETTER E APPEARS EVERYWHERE. EVERY TEAM MEMBER SHOULD CHECK EACH COLUMN. SEEK THE MOST FRE-

QUENT SYMBOL AND USE IT TO RECOVER THE KEY. WHEN THE MESSAGE BECOMES CLEAR, ENTER THE ACCESS WORD

3. Access word

ECLIPSE